

week-6 Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <unistd.h>
6  #include <sys/stat.h>
7  #include <errno.h>
8
9  #define REQUEST_FIFO "/tmp/request_fifo"
10 #define RESPONSE_FIFO "/tmp/response_fifo"
11 #define SIZE 1024
12
13 int main()
14 {
15     int request_fd, response_fd;
16     char message[SIZE];
17     char response[SIZE];
18
19     /* Create named pipes */
20     if (mkfifo(REQUEST_FIFO, 0666) == -1 && errno != EEXIST)
21     {
22         perror("mkfifo request");
23         exit(EXIT_FAILURE);
24     }
25
26     if (mkfifo(RESPONSE_FIFO, 0666) == -1 && errno != EEXIST)
27     {
28         perror("mkfifo response");
29         exit(EXIT_FAILURE);
30     }
31
32     printf("Server started...\n");
33     printf("Waiting for client messages...\n");
34
35     while (1)
36     {
37         /* Open request FIFO for reading */
38         request_fd = open(REQUEST_FIFO, O_RDONLY);
39
40         if (request_fd == -1)
41         {
42             perror("open request FIFO");
43             exit(EXIT_FAILURE);
44         }
45
46         /* Read message from client */
47         ssize_t n = read(request_fd, message, SIZE - 1);
48
49         if (n > 0)
50         {
51             message[n] = '\0';
52
53             printf("Client message: %s\n", message);
54
55             /* Process the message */
56             if (strcmp(message, "hello") == 0)
57             {
58                 strcpy(response, "Hello! Server received your message.");
59             }
60             else if (strcmp(message, "time") == 0)
61             {
57             strcpy(response, "Server is running successfully.");
62             }
63             else if (strcmp(message, "exit") == 0)
64             {
65                 strcpy(response, "Server shutting down.");
66             }
67
68             /* Send response */
69             response_fd = open(RESPONSE_FIFO, O_WRONLY);
70
71             WriteOut
72             Justify
73
74             Read File
75             Where is
76
77             Prev Pg
78             Next Pg
79
80             Cut Text
81             UnCut Text
82
83             Cur Pos
84             To Spell
85
86             Get Help
87             Exit
88
89             screenkhilreddy@Stress-MacBook-Pro ~ % nano kf.c
90             screenkhilreddy@Stress-MacBook-Pro ~ % gcc kf.c -o
91             clang: error: argument to '-o' is missing (expected 1 value)
92             screenkhilreddy@Stress-MacBook-Pro ~ % nano cp.c
93             screenkhilreddy@Stress-MacBook-Pro ~ % gcc cp.c -o cp
94             screenkhilreddy@Stress-MacBook-Pro ~ % ./cp
95             Server started...
96             Waiting for clients....
97             quit
98             is
99             exit
100

```

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

